

Editor comparison: tiptotyp, Texpile, Oleafly, and Typstify

This report records a feature comparison made on 20 September 2026. It is intended to distinguish product gaps from deliberate scope choices. Claims about Texpile and Oleafly come from their public documentation; absence from their documentation is not proof that a feature does not exist.

Executive summary

tiptotyp is currently a native, Typst-first source editor with unusually deep integration around Tinymist, preview recovery, Git hunks, multi-window state, themes, and performance diagnostics. Its biggest missing category is a visual or WYSIWYG editing mode. Texpile, Oleafly, and Typstify each cover more of the project-setup or visual-editing space in different ways.

Texpile is primarily a local document editor with collaboration and review features. Oleafly is a broader research workspace: references, literature search, AI-assisted tasks, templates, conversion tools, diagrams, and submission-oriented checks. Most Oleafly gaps are product-direction choices, not small editor features. Typstify is a cross-platform desktop editor that combines a conventional source editor and preview with package/template discovery, TPIX-backed project services, and optional power-saving behavior.

Typstify comparison

Typstify overlaps most directly with tiptotyp’s existing source-editor workflow. Its public documentation describes Typst and Tinymist integration, syntax highlighting, completion, hover and diagnostic tips, project folders, an outline, Git gutter markers, preview/export, and configurable external compiler/LSP paths. It also exposes features outside tiptotyp’s current scope: package and template browsing through TPIX, package publishing, dependency synchronization, Zotero-backed bibliography synchronization, and a power-saving mode that disables some Tinymist work.

The comparison is useful in both directions. Typstify validates package and template workflow as a high-value project feature, while tiptotyp currently has stronger evidence around native macOS behavior, explicit multi-window ownership, Git hunk actions, preview recovery, and bounded profiling. The most actionable Typstify-inspired gaps are package/template project actions, workspace outline polish, and an explicit low-resource mode. TPIX accounts, cloud services, package publishing, and bibliography synchronization should remain separate product decisions rather than being pulled into the editor core.

Capability comparison

Area	tiptotyp	Competitor capability	Gap or implication
Editing model	Source editor, Code/Split/Preview, Typst syntax tooling, and optional miTeX projection.	Texpile and Oleafly offer a visual prose editor that round-trips to .typ, .tex, or .md; unsupported constructs remain editable source blocks.	A true visual editor is the largest direct editing gap. It needs a lossless projection model, selection mapping, undo integration, and a clear performance budget.
Formats	Typst is first-class; text, images, PDFs, and miTeX are supported in narrower roles. Packaged builds pin Typst and Tinymist.	Both competitors advertise first-class LaTeX, Typst, and Markdown workflows. Oleafly also exposes several LaTeX engines and Pandoc-based paths.	Adding complete LaTeX or Markdown compilation would expand the toolchain and support burden substantially.
Editor productivity	Completion, hover, folding, diagnostics, find/replace, tabs, line comments, formatting, and configurable shortcuts.	Texpile documents a command palette, Vim and Emacs modes, multiple cursors, spell checking, and word/character counts.	Command palette, session restore, multiple cursors, semantic spell check, and counts are the most natural next productivity features.

Area	tiptotyp	Competitor capability	Gap or implication
Project workflow	Explorer, symbols, project index, packages, workspace root, Git status, staging, commits, fetch, pull, push, and hunk actions.	Texpile documents in-tree drag-and-drop, multi-select, search in files, contents navigation, and tab/session restoration.	Improve Explorer move/multi-select behavior and persist tabs, active tab, cursor, scroll, and preview selection.
Review and collaboration	Git diffs and hunk actions, but no comment threads or live collaboration.	Texpile offers comments with replies, resolve/filter behavior, and end-to-end encrypted real-time collaboration.	Comments and collaboration are a major scope increase and should not be mixed into ordinary editor refactoring.
Terminal and automation	No embedded terminal or general command palette.	Texpile has a built-in terminal; both products expose automation or AI integrations.	A terminal is already represented by todo item 170. A command palette is a smaller, high-value prerequisite for discoverability.
References and research	Project indexing and reference-aware completion, but no bibliography manager or literature workflow.	Oleafly provides citation/reference management, scholarly search, DOI and arXiv helpers, research folders, and research tasks.	A focused .bib index and citation completion could fit tiptotyp; a full research platform is a separate product direction.
PDF and publishing	Tinymist interactive preview, Poppler fallback, PDF/image tabs, and source navigation.	Oleafly advertises detached PDF viewing, richer SyncTeX, preflight checks, and export to PDF, DOCX, HTML, Markdown, and source ZIP.	PDF pop-out is already todo item 163. Preflight and broad export should be considered only after the core preview path is stable.
Templates and conversion	Package catalog recognizes template metadata, but there is no template gallery or conversion suite.	Oleafly offers research starters, templates, PDF/DOCX reconstruction, format conversion, and diagram tools.	Template discovery is a reasonable medium-sized feature; conversion and diagram generation are much larger integrations.
AI and diagrams	No built-in assistant, research task runner, or diagram composer.	Oleafly exposes optional model providers, local/CLI agents, AI edits, research tasks, and editable diagram generation.	Keep AI optional and outside the latency-sensitive editor path if this direction is chosen.

What tiptotyp already does well

- Native Rust and macOS integration with explicit multi-window ownership.
- Tinymist preview with retry-before-fallback behavior and a Poppler recovery path.
- Typst-aware completion, hover, folding, diagnostics, project indexing, package browsing, and source/preview navigation.
- Git status, staged and unstaged diffs, hunk navigation, and hunk actions.
- Bounded profiling scenarios, cache counters, process resource measurements, UI regression tests, and deterministic screenshot infrastructure.

- Pinned Typst/Tinymist sidecars and a large, configurable theme system.

These are not merely feature checkboxes: the explicit lifecycle, identity, and performance work gives tiptotyp a stronger foundation for a fast Typst-native editor than a broad feature list alone would show.

Recommended implementation order

The following order maximizes user value while keeping scope controlled:

1. Add a command palette over the existing AppCommand and shortcut registries.
2. Persist workspace session state: root, tabs, active tab, cursor, scroll, and preview-tab selection, with safe handling of deleted files.
3. Add selection-aware word and character counts.
4. Add semantic spell checking that skips Typst code, math, comments, and raw blocks.
5. Improve Explorer multi-select and in-tree drag-to-move.
6. Implement the built-in terminal from todo item 170.
7. Add a small .bib index and citation completion layer.
8. Implement PDF pop-out and multi-monitor behavior from todo item 163.
9. Only then decide whether a visual editor is central enough to justify a dedicated projection architecture.

Comments/collaboration, full LaTeX support, conversion, research search, AI, and visual diagram editing should be tracked as independent initiatives. They should not be smuggled into small cleanup tasks because each introduces new state, persistence, security, or toolchain boundaries.

Sources

- Texpile home and feature overview: <https://texpile.com/>
- Texpile documentation index: <https://texpile.com/docs>
- Texpile projects and files: <https://texpile.com/docs/projects>
- Texpile visual editing: <https://texpile.com/docs/visual-editing>
- Oleafly documentation overview: <https://oleafly.com/docs/>
- Oleafly product overview: <https://oleafly.com/>
- Typstify source repository and feature overview: <https://github.com/typstify/typstify>
- Typstify quick guide: <https://typstify.com/docs/quick-guide-v1.5>
- Typstify user manual: <https://typstify.com/docs/user-manual-v1.7>

The comparison is also grounded in the current repository feature summary in `README.md` and the open architecture/product tasks in `todo.typ`.